Which model fits this data?

Training-free model-family selection across 47 datasets

Tejaskumar Jaikrishnan
Lossfunk autovoila research track

Abstract. Choosing a model for a new dataset normally means training several and comparing them, which costs exactly the effort the choice was meant to save. We ask whether properties measurable *before* training can answer it instead. Across 47 datasets spanning 13 domains, evaluated under one prediction task with eight method families, the architecture is a second-order effect: choosing the best method over the second-best buys a median of 0.038 R^2, while the achievable ceiling varies by 1.232 across datasets, a factor of 33. Two things follow. First, that ceiling is itself predictable without training: an estimated noise floor correlates with the best achievable R^2 at $\rho = -0.87$. Second, a model-family selector built on seven such properties beats every fixed strategy and holds up when entire domains are held out (mean regret 0.038 against 0.063 for the best fixed choice, $p = 0.013$). The practical motivation is a finding we did not anticipate: neural networks do not degrade gracefully. On 18 of 47 datasets they land within 0.05 of the best available method; on 16 they fall below $R^2 = -0.5$, worse than predicting the training mean. Which side you land on is predictable from rows-per-dimension and coverage before a single gradient step. Controlled experiments then separate two failures that are usually conflated: what caps achievable accuracy, and what destroys the model you fitted.

1. Introduction

The standard way to pick a model is to fit several and keep the best. That is defensible when training is cheap and honest when the comparison is fair, but it inverts the usual order of enquiry: we learn about the data by spending compute on models, rather than deciding how much compute the data deserves. This paper asks whether the second order is available.

We frame the question as model-*family* selection. Given a dataset never seen before, and only statistics computable without fitting anything, which family of method should be reached for — a trivial baseline, a classical statistical method, or a neural network? We answer it with a selector validated out of sample, and scored by regret rather than accuracy, since recommending a method that was very nearly as good as the winner is not a real cost.

Our contributions are: (i) a quantification, across 47 datasets and 13 domains under one task, of how much the choice of method matters relative to the choice of dataset; (ii) evidence that the achievable ceiling is predictable from training-free properties; (iii) a model-family selector that transfers across held-out domains; (iv) the observation that neural networks in this regime either match the best method or fail catastrophically, with little in between; and (v) controlled experiments isolating three distinct ways a dataset defeats a model — irrelevance, non-stationarity, and stale delivery — only the first of which is visible in the achievable ceiling.

2. Setup

Data. 47 datasets across 13 kinds, listed in Table 1. Three are controls whose answers are known in advance, and they calibrate the pipeline: the best of all eight methods reaches $R^2 = -0.000$ on white noise and +1.000 on a periodic signal.

Kind	n	Median best R^2	Kind	n	Median best R^2
Simulated chaos	6	1.000	Biological	4	0.339
Controls	3	0.994	Sensor	5	0.316
Physical	3	0.886	Chemical	4	0.140
Economic	5	0.669	Engineering	1	0.082
Weather	3	0.656	Audio	1	–0.001
Human	3	0.622	Biomedical	8	–0.002
			Image	1	–0.006

Table 1: The 47 datasets by kind, with the best R^2 any of the eight methods achieved. Simulated physics is perfectly predictable; real biomedical and audio panels are not predictable at all.

Task. One task everywhere, so nothing is confounded by framing. Standardise, take a window of the last eight observations, predict the next. Train and test are contiguous blocks separated by a gap of 5% of the series, so nothing adjacent in time leaks across the split.

Methods. Eight families spanning what practitioners actually reach for: the training-set mean and persistence (the floors); ridge regression, k-nearest-neighbour analogue lookup, and gradient boosting (classical); and an MLP, a 1-D CNN and a GRU (neural). Three seeds each, 1,128 finite results.

Properties. Seven quantities computed without training: coverage (median distance from a test point to its nearest training neighbour, over the spread of the training set); an estimated noise floor (near-identical inputs whose next values disagree); distribution shift between train and test; decorrelation time; spectral entropy; dimension; and length.

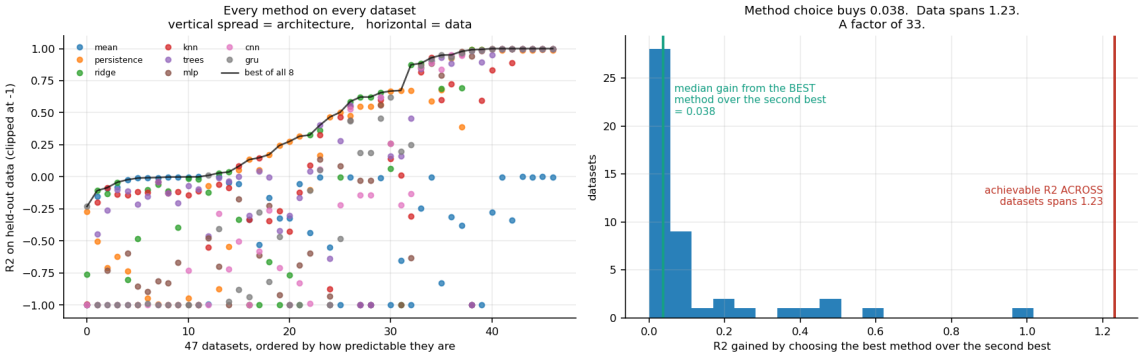

Figure 1: Every method on every dataset, ordered by predictability. Vertical spread is the architecture; horizontal spread is the data. Choosing the best method over the second best buys 0.038 R^2; the achievable ceiling spans 1.232.

3. Results

3.1 The choice of method is a second-order effect

Figure 1 shows all eight methods on all 47 datasets. The median gain from picking the best method rather than the second best is 0.038 R^2; the best achievable R^2 ranges from –0.232 to +1.000, a span of 1.232. The data therefore accounts for roughly 33 times more variation than the method does.

Counting winners makes the same point differently. Ridge regression is the most frequent winner at 14 of 47; persistence wins 10 and the training mean 8, so the trivial baselines together take 18. A neural network wins on 9.

One split must be made before any of this is read as a claim about architectures. Nineteen of the 47 datasets are tabular panels with no genuine time axis, included deliberately to see whether the pipeline would notice. It does: median best R^2 is 0.029 for those against 0.880 for the 28 genuinely temporal datasets, and a neural network wins 1 of 19 against 8 of 28. Asking a sequence question of data with no sequence yields nothing, correctly, and those datasets are excluded from any architectural conclusion.

3.2 Neural networks tie or fail, rarely in between

Across all 47 datasets the best neural network lands within 0.05 R^2 of the best available method on 18, and below R^2 = −0.5 — worse than predicting the training mean — on 16. Only 13 sit between. The distribution is bimodal, not graded.

The near-ties are genuinely near. On weekly CO_2 the network reaches 0.984 against ridge's 0.992; on monthly sunspots 0.867 against 0.886. It did not fail; it simply did not beat a linear model, at considerably greater cost. The failures, by contrast, are total: R^2 = −1.000 on *ecoli* (335 rows) and −0.931 on *wine* (178 rows), where a trivial baseline reaches 0.276 and 0.467.

Which side a dataset falls on tracks its size. Above 5,000 rows the median gap to the best method is 0.000 and one of nine datasets blows up; below 1,500 rows it is close to a coin flip. Rows per dimension separates the two groups more sharply still: a median of 274 where the network survived against 81 where it inverted. Coverage predicts the gap at ρ = +0.62.

This is the practical motivation for the rest of the paper. A method that degrades gently is safe to try; one that goes from tying the best to worse than the mean, with no warning, is not.

3.3 The achievable ceiling is predictable without training

For each dataset we take the best R^2 achieved by any of the eight methods and correlate it against each training-free property.

Property	**Spearman ρ**	**p**
Estimated noise floor	−0.871	< 0.0001
Spectral entropy	−0.739	< 0.0001
Decorrelation time	+0.662	< 0.0001
Coverage	−0.461	0.001
Dimension	−0.403	0.005
Length (rows)	+0.356	0.014
Distribution shift	+0.072	0.63

Table 2: Training-free properties against the best R^2 any method achieved, 47 datasets.

The noise-floor estimate alone accounts for most of it. Decorrelation time is the sharpest categorical signal: a decorrelation time of one step — consecutive rows independent — appears on every dataset where nothing works, regardless of method. Distribution shift is the exception and is treated separately in Section 3.6.

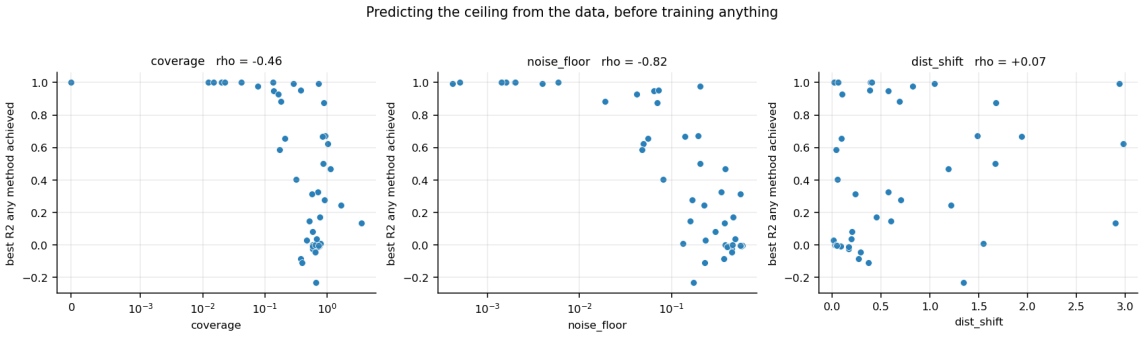

Figure 2: Three of the seven properties against the best R^2 any method achieved. All are computed without fitting a model.

3.4 A selector that transfers across domains

We fit a random forest to predict which of the three families achieves the best R^2, using only the seven properties. It is scored by regret — the R^2 lost against the best method actually available — under two validation schemes: leave-one-dataset-out, and the far harsher leave-one-domain-out, in which every dataset of a kind is held out together.

Strategy	Regret (dataset held out)	Regret (domain held out)
Oracle (unreachable)	0.000	0.000
Selector, from data properties	0.040	0.038
Always trivial	0.063	0.063
Always classical (majority)	0.119	0.119
Random choice	0.201	0.201
Always neural	0.419	0.419

Table 3: Mean regret, lower is better. The selector beats the best fixed strategy by 1.6× and always-neural by 11×.

The result that matters is that domain-holdout regret (0.038) is no worse than dataset-holdout regret (0.040). This was our pre-registered failure condition: had the selector only worked because it had seen four other chaotic systems, it would have learned the collection rather than a property of data. Paired Wilcoxon tests against the fixed strategies give p = 0.013 (always trivial), 0.006 (always classical) and < 0.0001 (always neural).

Family accuracy is 60% against a 33% chance baseline, which is modest; regret is the honest metric and is where the result lives. The winning method's identity also flips across seeds on 21 of 47 datasets, so this is a regime recommendation rather than a per-dataset oracle.

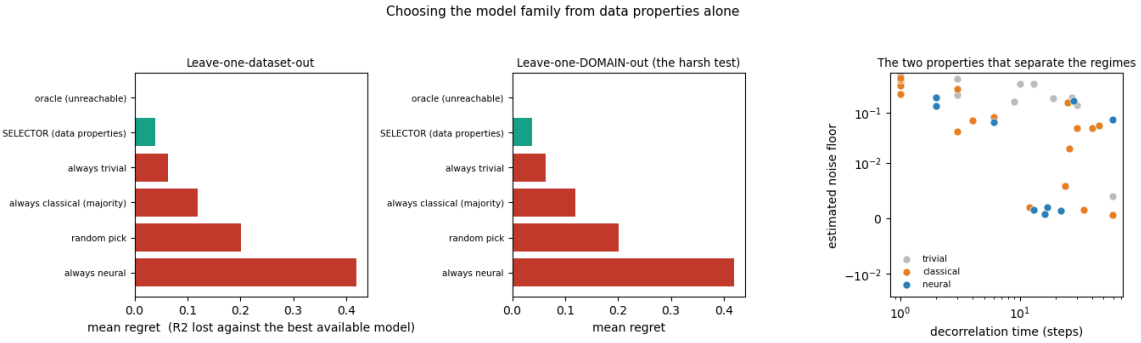

Figure 3: Regret against fixed strategies. The middle panel is the leave-one-domain-out test. The right panel shows the two properties that separate the regimes most cleanly.

The learned rule is legible. A depth-3 tree splits first on the noise floor, then on coverage, then on spectral entropy or distribution shift. Feature importances are spread — coverage 0.21, noise floor 0.18, dimension 0.14, rows 0.14, spectral entropy 0.14, shift 0.11, decorrelation 0.09 — which is why single correlations look weak while the combination does not.

3.5 What the three regimes look like

Winning family	n	Decorr. time	Coverage	Noise floor	Dims	Rows
Neural	9	16	0.077	0.064	3	6000
Classical	20	9	0.430	0.063	3	1912
Trivial	18	2	0.747	0.392	9.5	396

Table 4: Median data properties by which family wins. Neural networks win on smooth, densely covered, low-noise, plentiful data; trivial baselines win where coverage is sparse, dimension high and rows few.

Read together with Table 1, the picture is uncomfortable for the field. The regime where neural networks win is the regime where the data is nearly perfect — densely sampled, essentially noiseless, stationary and abundant. Four of the seven non-degenerate neural wins are our own simulated chaotic systems. Only three non-simulated datasets in the collection are won by a neural network.

3.6 What breaks a model: irrelevance and non-stationarity

Distribution shift is the one property in Table 2 that does not predict the ceiling (p = 0.63). To find out what it does instead we ran a controlled experiment: hold a three-variable chaotic system fixed and add only irrelevant columns, of four kinds — independent noise; slow smooth ramps; a second independent chaotic system; and lagged copies of the real variables.

Three results. First, **analogue lookup collapses far faster than a trained network**: adding 96 noise columns raises its one-step error from 0.019 to 0.640 while the network goes 0.001 to 0.112, because nearest-neighbour distance is computed over every column and junk dominates it. Second, **dimension alone is not the problem**: lagged copies, which add columns without adding confusion, cost almost nothing across a 32-fold dimension increase. Third, and most consequential, **the damaging kind of irrelevant column is the non-stationary one**. The smooth ramps shift by 0.83 of their own standard deviation between train and test, against 0.01–0.03 for every other kind; they raise the network's error by a factor of 896 and produce a generalisation gap of 38,415, while leaving the lookup baseline untouched.

Non-stationarity therefore does not lower what is achievable; it destroys the model that was fitted. These are different failures requiring different responses, and only the first is a property of the data. Figure 4 shows what each looks like in free-running prediction.

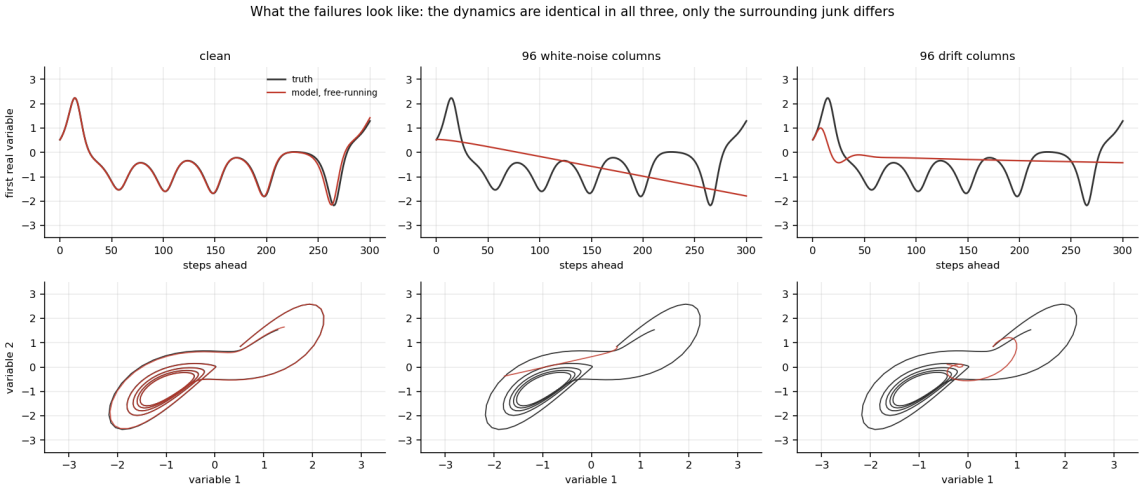

Figure 4: Free-running predictions against truth. The dynamics are identical in all three columns; only the surrounding irrelevant columns differ. Clean data tracks for 300 steps and reproduces the attractor; noise dilution decays along a straight line; non-stationary dilution flatlines almost immediately.

3.7 Delivery: a shifted signal survives, a frozen one does not

A second controlled experiment varies how the same information arrives. Four schemes carrying identical dynamics: no lag; a uniform lag of L steps; an intermittent scheme refreshing every L steps and holding between; and a jittered scheme giving every variable its own independent lag.

Two results run against intuition. **Jittered delivery is the most robust, not the least** — skill 0.730 at a 64-step spread where uniform lag has fallen to 0.276 — because a lag drawn from 0 to L leaves some channels nearly fresh, and partial freshness beats uniform staleness. And **intermittent delivery is far worse than uniform lag despite being less stale on average**: skill collapses to 0.010 at L = 64 where uniform lag holds 0.276. Holding a value destroys the signal's motion; a time-shifted signal keeps its velocity, a frozen one does not.

Adding an eight-step history window separates the two cleanly. It recovers +0.384 skill for intermittent delivery at L = 8, where it lets the model locate itself in the refresh cycle, but only +0.02 to +0.09 for uniform lag. Variable staleness is therefore a representation problem, cheap to fix; uniform delay is an information problem, and no architecture addresses it.

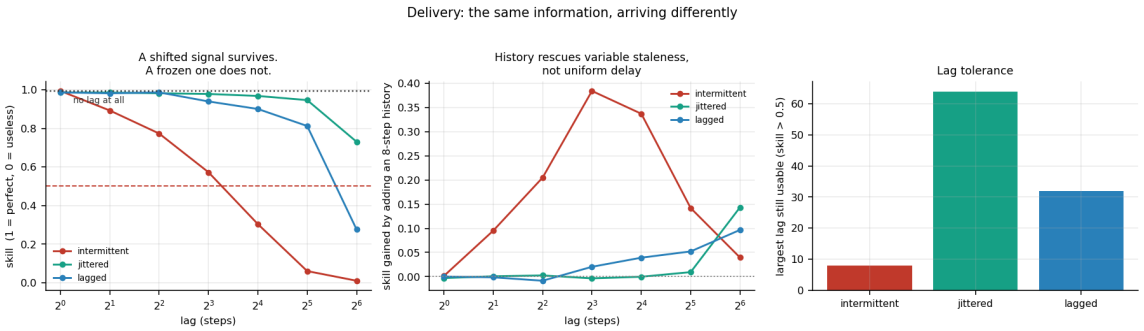

Figure 5: Skill against lag for three delivery schemes (left), the skill recovered by adding a history window (centre), and the largest lag each scheme tolerates (right).

3.8 A structural limit: locality

The results so far concern data. One result concerns architecture and is worth separating because it is arithmetic rather than an empirical trend. In a companion study we built twenty cellular-automaton architectures, pairing four local mechanisms with five global ones, all matched to 13,000 parameters, and measured how far a single perturbation propagates in one update.

A purely local rule moves information exactly one cell per step: the share of influence escaping that cone is **exactly zero**, at any tolerance and any parameter count. On a 100×100 grid one corner cannot influence the opposite corner in fewer than about 100 updates, regardless of model size. Architectures with a global pathway reach the whole lattice in one step.

That reach, however, mostly does not pay. Silencing the global pathway at evaluation time — the same weights, one term zeroed — changes the error by less than 2% in 43 of 128 cells tested. The exception is a pathway containing a learned map between specific positions, which lifts error by 1.33× when neighbours carry no information and only 1.06× when they do. The global mechanism earns its parameters precisely where locality fails, and is close to inert where locality holds.

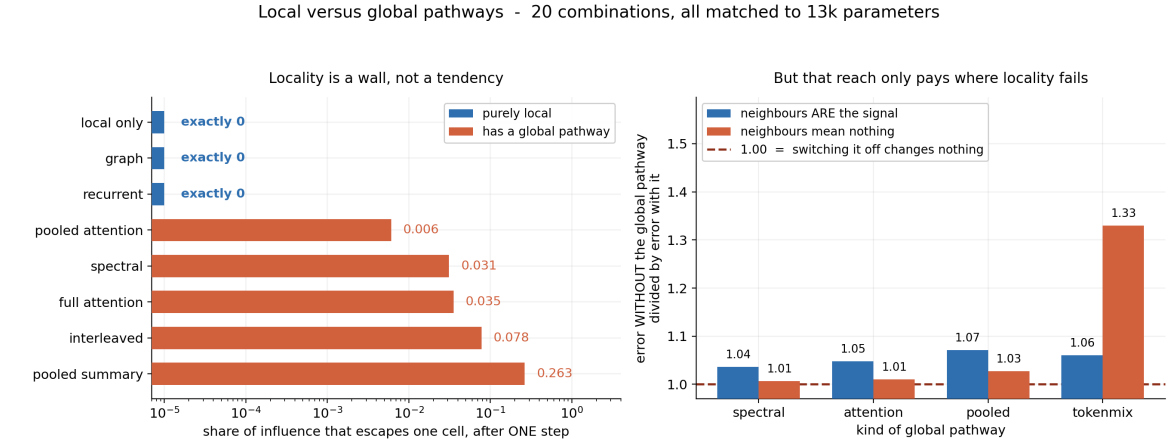

Figure 6: Left, the share of influence escaping one cell after one update; three architectures leak exactly zero. Right, the error penalty for switching the global pathway off, split by whether neighbours carry signal.

3.9 Stepping versus jumping straight to the horizon

A surrogate asked for the state at t+h can either take h single steps, feeding each output back in, or take the horizon as an input and jump there in one forward pass. The second avoids accumulating error by construction, so it is the natural test of whether long-horizon failure is caused by accumulation at all.

Across six chaotic systems and 360 runs, with the jumping model given four times the training pairs to remove the obvious handicap, stepping starts about 100× more accurate and stays usable roughly twice as long (to about 230 steps against 113, at a 0.4 error threshold). It reaches further on five of the six systems. Crucially the jump does *not* escape the long-horizon wall: beyond about 256 steps neither method is forecasting, with the jump settling onto the training average while stepping overshoots past it. Since a jumping model cannot accumulate error, what runs out at long horizons is information, not arithmetic precision.

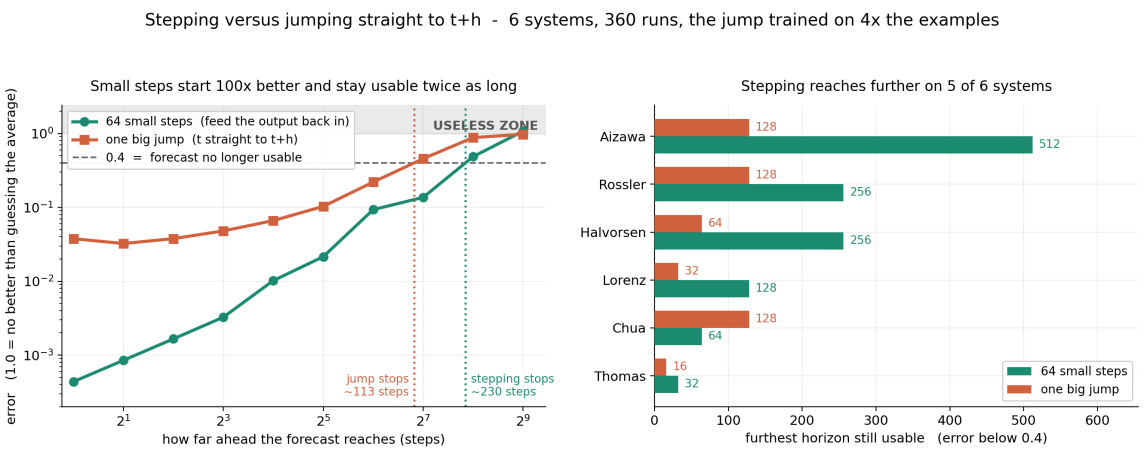

Figure 7: Error against forecast horizon for stepping and jumping (left), and the furthest usable horizon per system (right). Past the shaded region neither method forecasts.

4. Limitations

Scale. Every result was measured at 1–60 dimensions, 150–42,000 rows and 10^4–10^5 parameters, on one prediction task with a fixed window. The claim that architecture is second-order is measured only where every method is small, and is the claim most likely to invert at scale.

The noise-floor estimator degrades with dimension. Against synthetic data with a known ceiling of 0.80 it reads 0.80 at four dimensions, 0.68 at eight and 0.42 at 32, always pessimistically, because it requires near-identical inputs and in high dimensions nothing is near anything. Above roughly 16 dimensions it should be read as a lower bound. This is our most useful single result and its most exposed one.

Selector size. The selector is fitted on 47 points. Domain holdout is the strongest available test at that size, but it is still 47.

Not tested. Multi-task, action-conditioned and representation-learning settings. A model whose predictions influence the data it later sees is outside everything measured here, and is precisely the setting a world model occupies.

5. Related work

That simple baselines often match or beat deep models on time series is well established, from the M-competitions through to linear models outperforming transformers on standard forecasting benchmarks. Our contribution is not that observation but the question downstream of it: *which measurable property of the data decides it*, and does that property transfer to a domain the predictor has never seen.

Analogue forecasting, and the classical observation that natural analogues become vanishingly rare as dimension grows, supplies the mechanism behind our coverage measure and predicts the collapse of lookup under irrelevant columns reported in Section 3.6. Work on dataset difficulty and learning-curve extrapolation shares our target but generally requires partial training; the properties used here require none. Work relating forecast error to dynamical invariants such as the largest Lyapunov exponent is complementary: in a related study on 108 chaotic systems we found that exponent carries essentially no information about surrogate skill ($\rho = +0.03$), which is consistent with the present finding that the properties that matter are properties of the sample rather than of the underlying system.

6. Conclusion

On this evidence the dominant practical decision is not which network to build but whether to build one, and that question is answerable in seconds from the data. The ceiling is predictable at $\rho = -0.87$ without training; family selection beats every fixed strategy and survives domain holdout; and the reason it is worth doing is that neural networks in this regime fail catastrophically rather than gracefully.

The controlled experiments add a distinction we think is underused. A dataset can defeat a model in at least three separable ways: the answer may not be in the input (noise floor, which caps everyone); the relevant columns may be diluted by irrelevant ones (which hurts retrieval far more than learning); or the world may move between training

and deployment (which leaves the ceiling untouched and destroys the fitted model). Only the first is visible in a conventional error metric.

Two recommendations follow for practice. Report a training-free baseline alongside any surrogate result — across 47 datasets a trivial or classical method won 38 times. And report the data regime alongside the error, because two results with the same R^2 in different regimes have not demonstrated the same thing.

Reproducibility

All code, raw per-run results and figures are in the project repository. Every experiment writes one JSONL record per unit of work carrying its configuration hash and git commit, so any number here traces to a run. Datasets are fetched from public mirrors and cached; failures are reported rather than dropped, so a stated dataset count matches what ran. Total compute for the main study is 80 minutes on two CPU cores; the companion architecture study is a further four hours on the same hardware.

AI involvement

This work was carried out with an AI assistant (Claude) acting as the research agent under human direction. The assistant wrote the experiment code, ran the experiments, produced the figures and drafted this paper. The human author supplied the problem statement, chose the research direction at each decision point, supplied the reference material, and directed several changes of course — including the reframing from architecture comparison to model selection that this paper reports. All numerical claims were produced by code in the repository rather than generated by the model. Seven measurement artefacts were found and corrected during the work, four of them in our own instruments, and three attractive hypotheses of our own were discarded after controlled tests contradicted them; each correction is recorded in the repository history alongside the result it changed.